

QA FLASH

Multi-Linguagem, Multi-Framework e Multi-Plataforma

Web · Mobile · API · Mainframe

Explicacao completa para tecnicos e nao-tecnicos

Novembro 2026 · github.com/NiltesteQA/qa-flash-ui-front

1 Explicacao para Nao-Tecnicos

Entendendo sem precisar ser programador

💡 A ANALOGIA DO OPERADOR DE FABRICA

Pense no QA Flash como um **operador de fabrica** que sabe operar varias maquinas. Ele nao importa qual maquina voce coloque na frente dele - ele se adapta e trabalha.

Plataforma

Explicacao leiga: E o tipo de maquina onde o robo trabalha.

Exemplos: Navegador (site), Celular (app), Terminal verde (banco), API (sistema interno)

Linguagem

Explicacao leiga: E o idioma que o robo fala para comandar a maquina.

Exemplos: Java (empresas), Python (simples), TypeScript (frontend), Ruby (legado)

Framework

Explicacao leiga: E a ferramenta que o robo segura para operar.

Exemplos: Selenium (classico), Cypress (moderno), Playwright (rapido), Appium (celular)

BDD

Explicacao leiga: E a lingua que todo mundo entende - do estagiario ao diretor.

Formato: Dado (preparar), Quando (executar), Entao (validar)

Resumo: O QA escolhe 3 coisas (Plataforma + Linguagem + Framework) e o robo gera o codigo pronto. As regras de qualidade sao sempre as mesmas, independente da escolha.

2 Plataforma WEB

Testar sites no navegador

EXPLICACAO LEIGA

E como o robo abre o Chrome ou Edge e clica nos botoes de um site. Ele digita, clica, rola a pagina e verifica se tudo aparece corretamente - igual um humano faria, mas automatico.

Opcoes disponiveis

Linguagem	Frameworks	Explicacao de cada Framework	Dependencias	Executor
Java	Selenium 4 ou Playwright	Selenium = classico, 20 anos de mercado. Playwright = moderno e rapido	Maven (pom.xml)	JUnit 5
Python	Selenium ou Playwright	Mesma logica do Java, mas em Python (mais simples de ler)	pip (requirements.txt)	Pytest
TypeScript	Cypress ou Playwright ou WebdriverIO	Cypress = querido dos frontends. Playwright = rapido. WebdriverIO = flexivel	npm (package.json)	Mocha
Ruby	Capybara ou Selenium	Capybara = simples e direto. Selenium = padrao do mercado	Bundler (Gemfile)	RSpec

Importante: Em todas as linguagens, o BDD (Gherkin) e identico. O que muda e como o codigo executa por tras.

3 Plataforma MOBILE

Testar apps de celular (Android e iOS)

EXPLICACAO LEIGA

E como o robo abre o app no celular e toca na tela igual um dedo humano. Ele desliza, toca, digita e verifica se o app responde corretamente.

Opcoes disponiveis

Linguagem	Frameworks	Explicacao de cada Framework	Executor
Java	Appium	Appium = controla o celular como se fosse um dedo tocando	JUnit 5
Python	Appium-Python-Client	Mesma logica, em Python	Pytest
TypeScript	WebdriverIO + Appium ou Detox	Detox = focado em React Native. WebdriverIO = generico	Mocha
Ruby	Appium_lib	Mesma logica, em Ruby	RSpec

Diferenca entre Web e Mobile

Web: Robo usa o navegador (Chrome). Seletor: data-testid
Mobile: Robo usa o emulador ou celular real. Seletor: accessibility-id

O que muda no codigo

A logica do teste e a mesma. O que muda e **como** o robo encontra o botao na tela - no navegador e por CSS, no celular e por accessibility-id.

4 Plataforma API (Postman)

Testar endpoints REST - manual e automatizado

EXPLICACAO LEIGA

E como o robo liga para outro sistema e verifica se a resposta esta correta. Em vez de clicar na tela, ele envia um pedido (requisicao) e checa se a resposta (status, dados) esta certa.

Modo MANUAL (QA confere)

1. Robo gera Collection .json do Postman
2. QA importa no Postman
3. QA clica em "Send"
4. QA confere o resultado na tela
5. QA aprova ou reprova

Para que serve: Explorar, debugar, entender a API

Modo AUTOMATIZADO (CI/CD)

1. Robo gera scripts de teste
2. Newman roda no CI/CD
3. Resultado automatico
4. Relatorio gerado sozinho
5. QA revisa o relatorio

Para que serve: Regressao continua, pipeline

Opcoes disponiveis

Linguagem	Framework 1 (codigo)	Framework 2 (Postman)	Executor
Java	RestAssured	Postman/Newman	JUnit 5
Python	Requests	Postman/Newman	Pytest
TypeScript	Supertest	Postman/Newman	Mocha
Ruby	HTTParty	Postman/Newman	RSpec

5 Plataforma MAINFRAME

Testar terminal 3270 + IMS

EXPLICACAO LEIGA

E como o robo opera aquela tela verde de banco (terminal antigo), navegando com ENTER e teclas PF7/PF3. Em vez de mouse, usa o teclado. Em vez de botoes coloridos, usa campos em posicoes fixas (linha 5, coluna 20).

Opcoes disponiveis

Linguagem	Frameworks	Explicacao de cada Framework
Java	EZ3270 ou JZos	Controla o terminal verde via Java
Python	py3270 ou tn3270	Controla o terminal verde via Python
TypeScript	x3270 via Node.js	Controla o terminal verde via Node
Ruby	tn3270 gem	Controla o terminal verde via Ruby

Diferenca chave: Web vs Mainframe

Web: Acha o botao por nome/CSS
Mainframe: Acha o campo por **linha + coluna**

Diferenca chave: Acao

Web: Click, type, scroll
Mainframe: ENTER, PF3 (sair), PF7 (subir), PF8 (descer)

6 Matriz Completa de Combinacoes

Tudo que o robo suporta em uma tabela

EXPLICACAO LEIGA

Esta tabela mostra todas as combinacoes possiveis. O QA escolhe uma linha e o robo gera o codigo correspondente. Sao mais de 16 combinacoes diferentes.

Plataforma	Linguagem	Framework 1	Framework 2	Framework 3	BDD	Executor
Web	Java	Selenium 4	Playwright	—	Cucumber	JUnit 5
Web	Python	Selenium	Playwright	Behave	Behave	Pytest
Web	TypeScript	Cypress	Playwright	WebdriverIO	Cucumber	Mocha
Web	Ruby	Capybara	Selenium	SitePrism	Cucumber	RSpec
Mobile	Java	Appium	—	—	Cucumber	JUnit 5
Mobile	Python	Appium	—	—	Behave	Pytest
Mobile	TypeScript	WebdriverIO	Detox	—	Cucumber	Mocha
Mobile	Ruby	Appium_lib	—	—	Cucumber	RSpec
API	Java	RestAssured	Postman	—	Cucumber	JUnit 5
API	Python	Requests	Postman	—	Behave	Pytest
API	TypeScript	Supertest	Postman	—	Cucumber	Mocha
API	Ruby	HTTParty	Postman	—	Cucumber	RSpec
Mainframe	Java	EZ3270	JZos	—	Cucumber	JUnit 5
Mainframe	Python	py3270	tn3270	—	Behave	Pytest
Mainframe	TypeScript	x3270	—	—	Cucumber	Mocha

7 O Que Nao Muda em Nenhuma Combinacao

A base de qualidade que e sempre igual

EXPLICACAO LEIGA

Independente da linguagem ou framework escolhido, estas regras sao **sempre iguais**. E o que garante qualidade em qualquer projeto, seja Web, Mobile, API ou Mainframe.

BDD (Gherkin)

Dado, Quando, Entao - sempre em portugues, sempre igual. O diretor e o estagiario entendem a mesma coisa.

Padrao AAA

Arrange (Preparar), Act (Executar), Assert (Validar). Estrutura clara em todos os testes.

Page Object Model

Separar o mapeamento de tela do codigo de teste. Se a tela muda, so atualiza um arquivo.

Principios FIRST

Rapido, Independente, Repetivel, Autovailidado, Oportuno. Cada teste roda sozinho.

Locators Resilientes

Web: data-testid. Mobile: accessibility-id. Mainframe: linha+coluna. Nunca XPath absoluto.

Esperas Dinamicas

Nunca usar pausa fixa (Thread.sleep, cy.wait). Sempre esperar o elemento aparecer.

Seguranca

Credenciais sempre em .env, nunca em texto puro. Compliance OWASP e LGPD.

Human-in-the-Loop

4 paradas obrigatorias. O QA humano decide em cada uma. O robo nunca avanca sozinho.

8 Mesmo Teste, 3 Frameworks Diferentes

O BDD nao muda, so muda o codigo por tras

EXPLICACAO LEIGA

O mesmo cenario de teste escrito em 3 frameworks diferentes. Repare que a logica e a mesma - so muda a forma de escrever o codigo, como falar a mesma frase em 3 idiomas diferentes.

O cenario BDD (igual para todos)

```
Dado que estou na pagina de login
Quando preencho o e-mail e clico em entrar
Entao o dashboard deve aparecer
```

Selenium (Java)

```
driver.findElement(By.cssSelector(
    "[data-testid='email']"))
    .sendKeys("user@test.com");

driver.findElement(By.cssSelector(
    "[data-testid='btn-entrar']"))
    .click();

wait.until(ExpectedConditions
    .visibilityOfElementLocated(
        By.cssSelector("[data-testid='dashboard']")));
```

Cypress (TypeScript)

```
cy.get('[data-testid="email"]')
  .type('user@test.com');

cy.get('[data-testid="btn-entrar"]')
  .click();

cy.get('[data-testid="dashboard"]')
  .should('be.visible');
```

Playwright (TypeScript)

```
await page.fill('[data-testid="email"]', 'user@test.com');
await page.click('[data-testid="btn-entrar"]');
await page.waitForSelector('[data-testid="dashboard"]', { state: 'visible' });
```

9 Como o QA Escolhe

O fluxo de selecao na pratica

EXPLICACAO LEIGA

Quando voce inicia uma automacao, o robo te faz 3 perguntas simples. Voce responde e ele gera o codigo pronto na combinacao que voce escolheu.



10 Resumo Executivo

Argumentos para o setor de sistemas

Argumento	Detalhe
1. Um robo, multiplas plataformas	Web, Mobile, API e Mainframe - 4 plataformas em um so robo
2. 4 linguagens suportadas	Java, Python, TypeScript e Ruby - o time escolhe a que domina
3. 12+ frameworks	Selenium, Cypress, Playwright, Appium, RestAssured, Postman, py3270 e mais
4. 16+ combinacoes	Mais de 16 combinacoes de plataforma + linguagem + framework
5. BDD sempre igual	Gherkin nao muda entre frameworks - reuso total de cenarios
6. Qualidade sempre igual	FIRST, DRY, AAA, POM, OWASP, LGPD - em qualquer combinacao
7. Postman integrado	Teste de API manual (QA confere) e automatizado (Newman no CI/CD)
8. Human-in-the-Loop	O QA humano decide em 4 paradas - o robo nunca avanca sozinho
9. Baixo custo	80% da arquitetura se reutiliza entre plataformas. Nao comeca do zero
10. Equipe nao reaprende	Conceitos de BDD e Gherkin sao os mesmos. So aprende a ferramenta nova

Decisao: O QA Flash nao e um robo para uma linguagem. E um robo poliglota que se adapta ao que o time precisa, mantendo o mesmo padrao de qualidade em todas as combinacoes.



QA Flash

Multi-Linguagem, Multi-Framework, Multi-Plataforma

4 Plataformas: Web, Mobile, API, Mainframe

4 Linguagens: Java, Python, TypeScript, Ruby

12+ Frameworks: Selenium, Cypress, Playwright...

16+ Combinacoes possiveis

1 padrao de qualidade: sempre o mesmo

"Um robo poliglota. O QA escolhe, o robo executa, a qualidade nao muda."

Nilson Correa Souza

Analista de Teste - QA Engineer

Novembro 2026 · github.com/NiltesteQA/qa-flash-ui-front